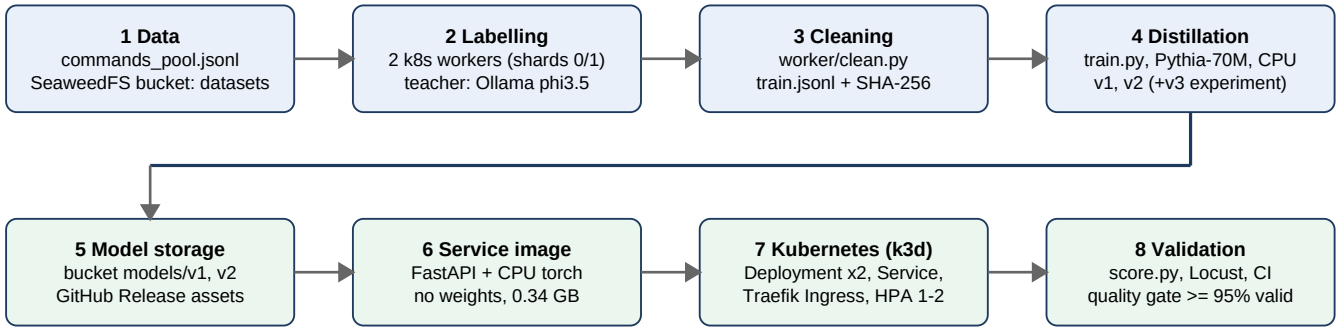
Knowledge Distillation and Deployment of a Command-Parsing Language Model on a Local Cloud

CS3081 Cloud Computing - Mini Project | Royal Sachan | Roll no. 230101021 |
https://github.com/royal-sachan/cloud-command-parser

Summary. A voice-assistant command parser turns one English command into JSON with four fields (intent, date, time, place; 8 intents). A larger *teacher* (Phi-3.5, 3.8B, served by Ollama) labelled 3,000 commands; after cleaning, 2,616 records remained and the first 1,500 were used to fine-tune a 70M-parameter *student* (Pythia-70M) by sequence-level distillation on CPU. The student returned valid JSON for 100% of the 100 validation commands and reached 40-46% full-frame accuracy (teacher 44%). Student v1 is packaged in a CPU-only Docker image (0.34 GB) that downloads its weights from SeaweedFS, and runs on a single-node k3d cluster behind a Traefik Ingress. No request failed in any of the recorded Kubernetes experiments.

How to read this report. [MEASURED] values were produced by my runs and are stored in *results/*; [DESIGN] describes how something is built; [ESTIMATE] is a calculation or assumption (not a measurement); [OPEN] marks work that is not completed. The validation set has only 100 commands, so differences of a few points are not statistically reliable.

Data flow of the project (all components run locally on one Windows laptop)



Results at a glance

Area	Result	Area	Result
JSON validity (student v1)	100% (teacher 98%)	Rolling update under load	0 / 1,338 requests failed; rollback showed v1
Full-frame accuracy v1 / v2 / v3	40% / 41% / 46% (teacher 44%)	Pod deleted under load	0 / 861 failed; 2 Ready replicas restored
Container image	0.34 GB (limit 2 GB)	Cold start (6 Pod readings)	median 8 s (range 7-11 s)
50 requests at 1 CPU / 1 GiB	50 x HTTP 200, peak 338 MiB, no OOM kill	Load balancing (2 replicas)	301 / 304 requests (49.8% / 50.2%)
Final load test (4 users, 2 min)	460 requests, 0 failed, p95 1,900 ms, 3.92 req/s	Estimated cost	0.56 INR per 1,000 requests (rate-card estimate)

1. Object storage and parallel teacher labelling

[DESIGN] A k3d cluster is created from *cluster/k3d.yaml*. SeaweedFS runs in namespace *data* with buckets *datasets* and *models* (S3 on localhost:8333); *commands_pool.jsonl* is uploaded to *datasets*. The teacher is Phi-3.5 on Ollama (temperature 0, seed 42, JSON-constrained output, at most 100 new tokens) running on the laptop; the prompt is *prompts/teacher_prompt.txt* (instruction, the 8 intents, the 3 slot rules and 8 hand-written examples). Two workers run at the same time as an Indexed Job in namespace *pipeline* and reach Ollama through *host.docker.internal*.

Sharding. Each worker reads *SHARD_INDEX* and *SHARD_COUNT* and takes the commands whose line number n (counted from 0) satisfies n mod SHARD_COUNT = SHARD_INDEX. Every line therefore belongs to exactly one shard (no gaps or repeats) and the shards are equal in size without any coordination between workers. Each worker writes its own object, *datasets/teacher/shard-<i>.jsonl*, containing the id, the command and the teacher's exact reply (never edited). Progress is saved every 20 records and a restarted worker skips ids already done, which matters because CPU labelling is slow. [MEASURED] Labelling was interrupted once, was resumed, and finished with 1,500 records in each shard. Because one CPU serves a single Ollama instance, the second worker mainly demonstrates the pipeline design; throughput is limited by the teacher.

Input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) bad slot	Duplicates	Kept
3,000	6	55	6	274	43	2,616

[MEASURED] Check: 6 + 55 + 6 + 274 + 43 + 2,616 = 3,000. 87.2% of replies were kept. The largest rejection reason was (d): 274 of the 341 non-duplicate rejections had a slot value that did not appear verbatim in the command, which breaks the copy-exactly rule. *clean.py* applies (a)-(d) per record, then removes duplicate command texts (keeping the smallest id), writes *datasets/v1/train.jsonl* in id order as LF-only bytes, and stores the SHA-256 in *train.sha256*, so the checksum is the same on Windows and Linux.

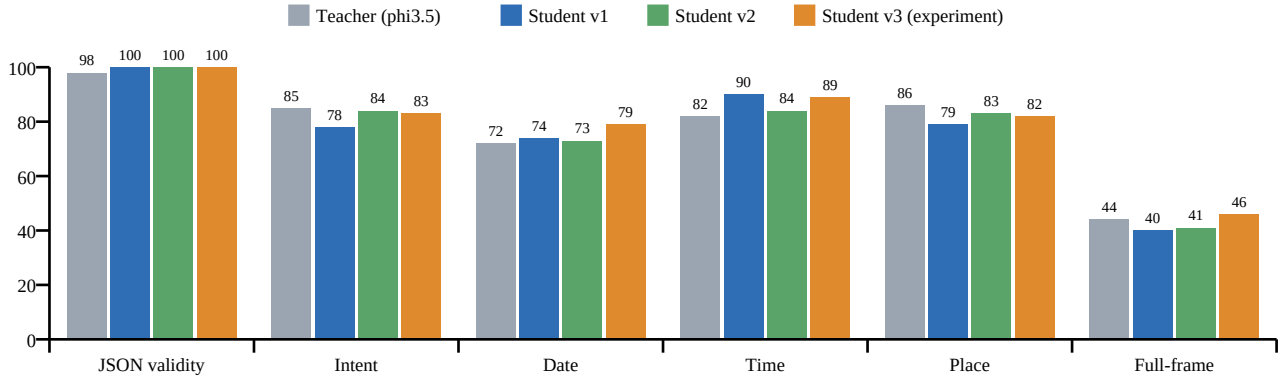
2. Training the student (sequence-level distillation)

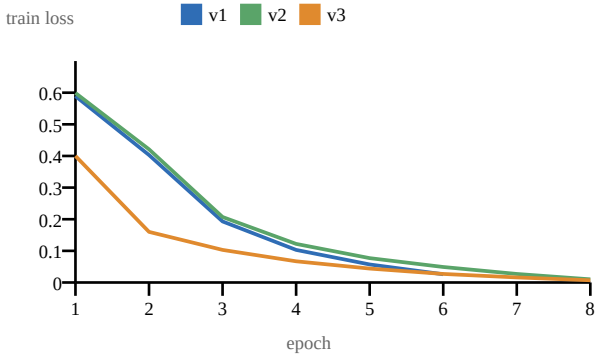
[DESIGN] Pythia-70M is fine-tuned with PyTorch and Hugging Face Transformers on CPU using the first 1,500 cleaned records. Each example is *Command: ...* newline *JSON: ...* plus an end-of-sequence token; the loss is applied to the JSON part only. The teacher reply is re-serialised with a fixed key order (same content) so the target format is consistent. *make train* reads only *training/config.yaml*; *make train SUBSET=50* trains on the first 50 records as a smoke test (it ran to the end). The loss of every epoch is stored in *training/loss.csv*.

Version	Seed	Learning rate	Batch	Epochs	Max length	Status
v1 (deployed)	42	3e-4	16	6	96	weights in bucket models/v1 and GitHub Release
v2 (rolling-update target)	7	3e-4	16	8	96	weights in bucket models/v2 and GitHub Release
v3 (extra experiment)	42	1e-4	16	8	96	evaluated only; not deployed, no release

Metric (%), 100 validation commands	Teacher phi3.5	Untrained Pythia-70M	Student v1	Student v2	Student v3
JSON validity	98	0	100	100	100
Intent accuracy	85	0	78	84	83
Date accuracy	72	0	74	73	79
Time accuracy	82	0	90	84	89
Place accuracy	86	0	79	83	82
Full-frame (all four fields correct)	44	0	40	41	46

[MEASURED] Scores from *tools/score.py*; reports and raw outputs for the teacher, the untrained model and all students are in *results/*. Full-frame means intent, date, time and place are all correct for the same command.





Discussion. The untrained model never produced valid four-key JSON (0%): it has no notion of the task. After fine-tuning on 1,500 teacher replies all students were 100% valid, so the output format is learned quickly; choosing the right intent and copying the right slot text is harder for 70M parameters. The students land close to the teacher (full-frame 40-46% vs 44%) because they imitate it, mistakes included. Some student fields exceed the teacher's (e.g. time 90 vs 82), but with 100 examples such gaps are within noise.

On v3. v3 (lower learning rate) is 5 points above v2 on full-frame. I treat this cautiously: it was chosen after looking at these same validation scores, the set is small, and one seed was run, so it is not evidence of a general improvement. [MEASURED] Loss fell steadily for every run (left: v1 to 0.026, v2 to 0.010, v3 to 0.007); a low training loss alone does not show generalisation.

Sequence-level vs logit-based distillation. Sequence-level distillation (used here) trains the student with ordinary cross-entropy on the text the teacher generated, i.e. hard labels. Logit-based distillation trains the student to match the teacher's probability distribution over the next token (soft labels, e.g. a KL-divergence loss). It is not possible in this project because Phi-3.5 and Pythia use different tokenizers and vocabularies, so their per-token distributions do not line up, and only the teacher's text was stored, not its logits. **Limitation:** the student inherits the teacher's mistakes (the teacher itself reaches only 44% full-frame) and sees one decoded answer per command, with no information about the teacher's uncertainty.

[ESTIMATE] Task 2 hidden-test requirements are 95% JSON validity and full-frame at least half of the teacher's (22%). The validation margins (100% and 40%) are comfortable, but the hidden test set was not available, so I make no claim about it.

3. Inference service in a container

[DESIGN] *service/app.py* is a FastAPI service on port 8000. **POST /parse** takes *{"text": ...}* and returns only the four-key JSON object with headers *X-Model-Version* and *X-Served-By* (the Pod name). **GET /healthz** returns 200 while the process is alive; **GET /readyz** returns 503 until the model is loaded (loading runs in a background thread, so liveness is unaffected). Configuration comes only from *MODEL_URI, MODEL_VERSION, S3_ENDPOINT, MAX_NEW_TOKENS* (plus the optional *FAULT_INJECT*); S3 credentials

come from a Kubernetes Secret. At start-up the service downloads the model and tokenizer from SeaweedFS. Every output is checked against *data/schema.json*; if it is invalid the service returns HTTP 500 *{"error": "invalid_model_output"}* and never broken JSON with HTTP 200. With *FAULT_INJECT=1* the output is replaced by *not json* before the check. The image is built from python:3.11-slim with a CPU-only PyTorch wheel, every dependency pinned and no weights inside.

Check (docker, --cpus=1 --memory=1g)	Result	Requirement
Image size	0.34 GB	2 GB or less
50 sequential requests	50 of 50 returned HTTP 200	no failures
Peak memory / OOM kill	338 MiB of 1,024 MiB / not OOM-killed	no OOM kill; peak reported
FAULT_INJECT=1	HTTP 500, body invalid_model_output	exact error body

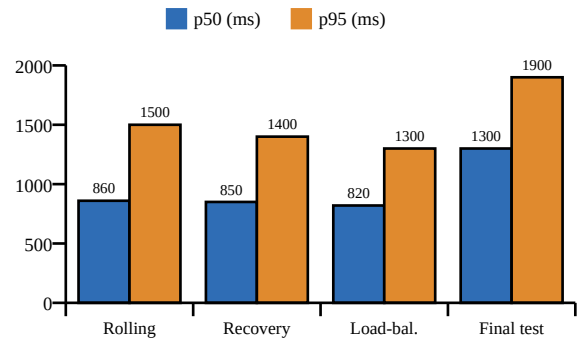
[MEASURED] Evidence in *results/task3_limits.txt* and *results/task3_fault_inject.txt*. The first run exposed a missing *accelerate* dependency; I added a pinned version and re-ran the test.

4. Kubernetes deployment, scaling and reliability

[DESIGN] In namespace *app*, a Deployment (2 replicas) runs the image with CPU/memory requests of 500m / 512Mi and limits of 1 CPU / 1 Gi, a liveness probe on /healthz and a readiness probe on /readyz. Rolling updates use maxUnavailable 0, maxSurge 1 and an 8-second preStop delay so traffic drains before a Pod stops. A Service and a Traefik Ingress route */parse*. A HorizontalPodAutoscaler keeps 1-2 replicas at 70% CPU utilisation, which is measured relative to the 500m request.

Experiment (standard Locust load, 4 users)	Requests	Failed	p50 ms	p95 ms	req/s	Outcome
Rolling update v1 to v2, then rollout undo	1,338	0 (0.00%)	860	1,500	4.49	v1: 1,115 and v2: 226 responses; header showed v1 after undo
One Pod deleted under load	861	0 (0.00%)	850	1,400	4.81	Ready replicas restored to 2
Load balancing, exactly 2 Ready replicas	601	0 (0.00%)	820	1,300	5.06	301 (49.8%) and 304 (50.2%) per replica
HPA test (1 to 2 and back)	1,134	0 (0.00%)	-	-	-	CPU reached 180-199% of target; 2 replicas, later 1

[MEASURED] Targets were at most 0.5% failures (rolling update) and at most 2% (recovery); both met. Logs: *results/rolling.log, recovery.log, lb.log, hpa.log* and the Locust CSV/JSON files.



Readiness. When the Deployment was scaled from 0 to 1 Pod, the Service listed no endpoints (*parser <none>*) while the Pod showed 0/1; an endpoint (10.42.0.42:8000) appeared only after the Pod became 1/1 Ready (*results/endpoints.log*).

Cold start. Three runs deleted every Pod; six Pods were created in total. Creation-to-Ready times were 8, 8, 11, 11, 7 and 7 s, median 8 s. The logs show *downloaded 8 model files from s3://models/v1/* followed by *model loaded, ready*. This is fast because the image is already on the node and SeaweedFS is on the same laptop; on a real cloud the image pull and download would add time (estimate).

Limitation. The cluster is one k3d node on one laptop. It demonstrates Pod self-healing, rolling updates and rollback, but not fault tolerance across machines: losing the laptop or Docker loses everything. Metrics-server also scrapes at a coarse interval, so the HPA reacts with a delay.

5. Subnetting and network access control

[DESIGN] The block 10.20.0.0/16 is split largest-first, each subnet placed right after the previous one with the smallest prefix that fits (usable hosts = 2^(32-prefix) - 2). Full working is in *network/subnets.md*.

Subnet	Hosts needed	Network / prefix	Usable range	Broadcast	Usable hosts
app	1,000	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

Direction	Rule	Action	Protocol / ports	Address
Inbound	100	deny	all	198.51.100.0/24 (abusive range)
Inbound	110, 120	allow	TCP 80, TCP 443	0.0.0.0/0
Inbound	130	allow	TCP 22	10.20.5.128/27 (management)
Inbound	140	allow	TCP 1024-65535 (replies)	10.20.0.0/22 (app)
Outbound	100	allow	TCP 8000	10.20.0.0/22 (app)
Outbound	110	allow	TCP 1024-65535 (replies to clients)	0.0.0.0/0

[MEASURED] *nacl_sim.py --check* decided all 6 sample packets correctly (6/6). Anything not matched is denied by default.

Why replies need their own rules. A network ACL is stateless: it keeps no record of connections, so a reply packet is judged as a new packet against the rules for its own direction. Clients use ephemeral ports 1024-65535, so the reply direction needs explicit rules for that range. **Why order matters.** Rules are checked from the lowest number upward and the first match wins. The deny for 198.51.100.0/24 is rule 100, below the allows for ports 80 and 443; if it had a higher number, a packet from that range to port 443 would match the allow first and get through.

6. Reproducibility, testing and CI/CD

[DESIGN] Everything (code, tests, training and evaluation scripts, manifests, network files, workflow) is in the GitHub repository. *make up* creates the cluster, SeaweedFS and buckets, restores the committed datasets, downloads the model Release assets into the bucket, builds and imports the image and deploys the app; *make down* deletes the cluster. The workflow *.github/workflows/ci.yml* runs, in order: lint (ruff), unit tests (pytest), model evaluation (downloads *model-v1.zip* from the Release and runs *score.py* on *validation.jsonl*) and a quality gate (*score.py --min-validity 95*); a gitleaks secret scan runs as a separate job. S3 keys are generated locally into a git-ignored folder and injected as Secrets, never committed.

Item	Status
Secret scan, lint, unit tests, model evaluation, quality gate with the trained model	[MEASURED] all passed: run 42460941563 (new repository)
Baseline run (untrained model) failing at the quality gate	[OPEN] no baseline run has been recorded yet. An earlier attempt did not start because the old GitHub account was billing-locked. The gate can only be shown failing by running the workflow with the untrained Pythia-70M model.
Fresh-clone test of make up	[OPEN] optional test was not performed
AI use disclosure	see AI_USE.md (Claude and Copilot used for explanations, drafts, scripts and report support; results checked against result files, lint and tests)

The imported Git history required a correction to the secret-scan workflow when the project moved to the new repository.

7. Performance, cost and review

p50	p95	Throughput	Requests / failures	Error rate	Peak CPU / memory per Pod (kubectl top)
1,300 ms	1,900 ms	3.92 req/s	460 / 0	0.0%	998m, 398 MiB and 952m, 338 MiB

[MEASURED] Standard command: 4 users, 2 minutes, through the Ingress; files *results/loadtest_** and *kubectl_top_pods.txt*. Cold-start time: see section 4 (median 8 s). Both Pods peaked near the 1-CPU limit. Latency here is higher than in the earlier load-balancing run (p50 820 ms); I did not isolate the cause (CPU throttling at the limit or other load on the laptop are possible, not tested).

Cost. [ESTIMATE] Using *rate_card.json* (3.5 per vCPU-hour, 0.45 per GiB-hour), each replica is billed for its limits (1 CPU, 1 GiB). Hourly cost of 2 replicas = 2 x (1 x 3.5 + 1 x 0.45) = 7.90 INR. Requests per hour = 3.919 x 3,600 = 14,108. Cost per 1,000 requests = 7.90 / 14,108 x 1,000 = **0.56 INR**. This is an estimate from a hypothetical rate card, not a cloud bill.

SLO. [MEASURED] Objective: 99% of requests succeed (HTTP 200 within 30 s); window: one 2-minute load-test run. Error budget = 1% x 460 = 4.6 requests; used 0 (0%).

Review table

Component	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	2,616 of 3,000 replies kept (87.2%); 274 rejected for bad slots	One SeaweedFS replica on one volume; teacher labels unreviewed	Replicated volumes; versioned datasets; filter labels by agreement
Model quality	v1 full-frame 40%, validity 100% (teacher 44% / 98%)	70M model, 1,500 examples, teacher errors copied, 100-example test	Larger held-out set; better labels; same-tokenizer teacher for logit distillation
Latency and throughput	p50 1,300 ms, p95 1,900 ms, 3.92 req/s	Greedy CPU decoding; one request at a time per Pod	Quantisation or ONNX; shorter output format
Resource utilisation	998m / 952m CPU, 398 / 338 MiB per Pod	CPU near the 1-core limit under 4 users	Tune requests and limits from measurements
Reliability and recovery	0 failures in 1,338 (rolling) and 861 (recovery) requests	Single node: no machine-level fault tolerance	Multi-node cluster; PodDisruptionBudget
Security and cost	0.56 INR per 1,000 requests (estimate); no secrets in Git	Plain Kubernetes Secrets; no TLS or auth on /parse	Sealed secrets; TLS on Ingress; request authentication

Open items and assumptions

(1) The baseline CI run that fails at the quality gate is not recorded (section 6). (2) The optional fresh-clone test was skipped. (3) The hidden test set was not available, so no hidden-test result is claimed. (4) v3 was not deployed or released; all Kubernetes experiments used v1 (v2 only as the rolling-update target). (5) Total labelling duration was not recorded.

References

Kim, Y. and Rush, A. M. (2016). Sequence-Level Knowledge Distillation. EMNLP 2016. https://aclanthology.org/D16-1139/

Chen, Y.-C. et al. (2020). Distilling Knowledge Learned in BERT for Text Generation. ACL 2020. https://aclanthology.org/2020.acl-main.705/

Bastianelli, E. et al. (2020). SLURP: A Spoken Language Understanding Resource Package. EMNLP 2020 (CC BY 4.0). https://aclanthology.org/2020.emnlp-main.588/

Biderman, S. et al. (2023). Pythia: A Suite for Analyzing Large Language Models. Model card: https://huggingface.co/EleutherAI/pythia-70m

Microsoft. Phi-3.5-mini-instruct, served with Ollama: https://ollama.com/library/phi3.5

k3d: https://k3d.io; SeaweedFS: https://github.com/seaweedfs/seaweedfs; Locust: https://locust.io; Kubernetes documentation: https://kubernetes.io/docs

Appendix: evidence map

Task	Where the evidence is
1	prompts/teacher_prompt.txt, data/teacher/shard-*.jsonl, data/train.jsonl, data/train.sha256, results/clean_counts.json
2	training/config.yaml, training/loss.csv, results/*_validation.json and *_predictions.jsonl (teacher, untrained, v1, v2, v3), GitHub Releases model-v1 and model-v2
3	service/, results/task3_limits.txt, results/task3_fault_inject.txt
4	k8s/, results/endpoints.log, rolling.log, recovery.log, coldstart.log, hpa.log, lb.log and Locust CSV/JSON files
5	network/subnets.md, network/nacl.json (nacl_sim.py 6/6)
6	.github/workflows/ci.yml, tests/, AI_USE.md, README.md
7	results/loadtest_*, results/kubectl_top_pods.txt, results/perf_summary.json